

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Операционные среды и системное программирование»

ОТЧЁТ
к лабораторной работе №4
на тему
«ОСНОВЫ ПРОГРАММИРОВАНИЯ НА С ПОД UNIX.»
БГУИР 6-05-0612-02 23

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

(дата, подпись студента)

Проверил ассистент каф. информатики
ГРИЦЕНКО Никита Юрьевич

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1	Постановка задачи	3
2	Выполнение работы	4
2.1	Архитектура программы	4
2.2	Обработка аргументов командной строки	4
2.3	Алгоритм фильтрации и работа с памятью	4
2.4	Сборка и тестирование	4
	Вывод	6
	Список использованных источников	7
	Приложение А (обязательное) Листинг программного кода	8

1 ПОСТАНОВКА ЗАДАЧИ

Разработать программу на языке C, реализующую фильтр автоматической цензуры. Программа должна читать входной поток данных (текст) и заменять все вхождения запрещенных слов на последовательность символов «***». Список запрещенных слов должен загружаться из внешнего текстового файла (словаря). Программа должна поддерживать работу как со стандартными потоками ввода/вывода (*stdin*, *stdout*), так и с файлами, указанными через аргументы командной строки.

2 ВЫПОЛНЕНИЕ РАБОТЫ

2.1 Архитектура программы

Программа реализована на языке *C* стандарта *C18* [1]. Архитектура построена по модульному принципу и включает следующие компоненты:

- Модуль *dictionary* отвечает за динамическое управление списком запрещенных слов. Словарь реализован в виде структуры с динамическим массивом строк для обеспечения эффективного хранения и поиска.
- Модуль *filter* содержит логику обработки текста. Выполняет разбиение строки на слова, проверку каждого слова по словарю и формирование отфильтрованного результата.
- Главный модуль *main* координирует работу программы, обрабатывает аргументы командной строки и организует цикл чтения данных.

2.2 Обработка аргументов командной строки

Для разбора параметров запуска используется стандартная функция *getopt* [2]. Программа поддерживает следующие флаги:

- *-d <path>* – путь к файлу словаря (обязательный параметр);
- *-i <path>* – путь к входному текстовому файлу (по умолчанию *stdin*);
- *-o <path>* – путь к выходному файлу (по умолчанию *stdout*).

При отсутствии флага *-d* или невозможности открыть указанные файлы программа выводит соответствующее сообщение об ошибке в поток *stderr* и завершает работу с ненулевым кодом статуса.

2.3 Алгоритм фильтрации и работа с памятью

Загрузка словаря выполняется построчно. Каждое слово из файла очищается от символов переноса строки и сохраняется в динамической памяти. Поиск в словаре осуществляется без учета регистра символов с помощью функции *strcasestr*.

Обработка основного текста выполняется построчно с использованием функции *getline* [3], что позволяет эффективно работать со строками произвольной длины. Функция фильтрации идентифицирует слова как последовательности алфавитных символов (*isalpha*). Знаки препинания, пробелы и цифры сохраняются без изменений. Для каждой обработанной строки выделяется новый буфер памяти, который освобождается после вывода результата.

2.4 Сборка и тестирование

Сборка проекта осуществляется с помощью утилиты *make*. В *Makefile* определены цели для компиляции исполняемого файла и очистки проекта.

Тестирование работоспособности программы проводилось вручную на различных наборах входных данных, включая передачу аргументов командной

строки для работы с файлами и использование стандартных потоков ввода/вывода. Программа корректно обрабатывает пустые файлы, длинные строки текста и сложные случаи пунктуации.

ВЫВОД

В ходе выполнения лабораторной работы была разработана программа фильтрации текста на языке *C*, реализующая механизм автоматической цензуры. В процессе реализации были закреплены навыки работы с динамическими структурами данных, системными вызовами для работы с файловой системой и средствами обработки аргументов командной строки. Особое внимание было уделено вопросам безопасного управления памятью и тестирования программного обеспечения в среде *Linux*. Программа демонстрирует стабильность работы и корректную обработку текстовых данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] ISO/IEC 9899:2018 - Information technology – Programming languages – C [Электронный ресурс]. – Режим доступа: <https://www.iso.org/standard/74528.html>. – Дата доступа: 17.03.2026.

[2] getopt(3) - Linux manual page [Электронный ресурс]. – Режим доступа: <https://man7.org/linux/man-pages/man3/getopt.3.html>. – Дата доступа: 17.03.2026.

[3] getline(3) - Linux manual page [Электронный ресурс]. – Режим доступа: <https://man7.org/linux/man-pages/man3/getline.3.html>. – Дата доступа: 17.03.2026.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программного кода

Листинг А.1 – main.c

```
#include "dictionary.h"
#include "filter.h"
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    int opt;
    char *dict_file = NULL;
    char *input_file = NULL;
    char *output_file = NULL;

    while ((opt = getopt(argc, argv, "d:i:o:")) != -1) {
        switch (opt) {
            case 'd':
                dict_file = optarg;
                break;
            case 'i':
                input_file = optarg;
                break;
            case 'o':
                output_file = optarg;
                break;
            default:
                fprintf(stderr,
                    "Usage: %s -d dict.txt [-i input.txt] [-o
output.txt]\n",
                    argv[0]);
                return 1;
        }
    }

    if (!dict_file) {
        fprintf(stderr, "Error: Dictionary file (-d) is required.\n");
        return 1;
    }

    FILE *df = fopen(dict_file, "r");
    if (!df) {
        perror("Failed to open dictionary file");
        return 1;
    }

    Dictionary *dict = dict_create();
    if (!dict) {
        fprintf(stderr, "Failed to create dictionary.\n");
        fclose(df);
    }
```

```

        return 1;
    }

    dict_load(dict, df);
    fclose(df);

    FILE *in = stdin;
    if (input_file) {
        in = fopen(input_file, "r");
        if (!in) {
            perror("Failed to open input file");
            dict_free(dict);
            return 1;
        }
    }

    FILE *out = stdout;
    if (output_file) {
        out = fopen(output_file, "w");
        if (!out) {
            perror("Failed to open output file");
            if (in != stdin)
                fclose(in);
            dict_free(dict);
            return 1;
        }
    }

    char *line = NULL;
    size_t len = 0;
    ssize_t read_bytes;

    while ((read_bytes = getline(&line, &len, in)) != -1) {
        char *filtered = filter_line(line, dict);
        if (filtered) {
            fputs(filtered, out);
            free(filtered);
        }
    }

    free(line);

    if (in != stdin)
        fclose(in);
    if (out != stdout)
        fclose(out);
    dict_free(dict);

    return 0;
}

```

Листинг А.2 – filter.c

```

#include "filter.h"
#include <ctype.h>
#include <stddef.h>

```

```

#include <stdlib.h>
#include <string.h>

char *filter_line(const char *line, Dictionary *dict) {
    if (!line || !dict)
        return NULL;

    size_t len = strlen(line);
    size_t cap = len * 3 + 1;
    char *result = malloc(cap);
    if (!result)
        return NULL;

    size_t i = 0;
    size_t res_i = 0;

    while (i < len) {
        if (isalpha((unsigned char)line[i])) {
            size_t word_start = i;
            while (i < len && isalpha((unsigned char)line[i])) {
                i++;
            }
            size_t word_len = i - word_start;

            char *word = malloc(word_len + 1);
            if (!word) {
                free(result);
                return NULL;
            }
            strncpy(word, line + word_start, word_len);
            word[word_len] = '\0';

            if (dict_is_forbidden(dict, word)) {
                for (size_t j = 0; j < word_len; j++) {
                    result[res_i++] = '*';
                }
            } else {
                for (size_t j = 0; j < word_len; j++) {
                    result[res_i++] = word[j];
                }
            }
            free(word);
        } else {
            result[res_i++] = line[i++];
        }
    }
    result[res_i] = '\0';

    char *shrunked = realloc(result, res_i + 1);
    if (shrunked) {
        result = shrunked;
    }
    return result;
}

```